The main objective of most attackers is to make money. The underground prices for stolen bank login accounts range from $10-$1000 (depending on the available amount of funds), $0.40-$20 for credit card numbers, $1-$8 for online auction site accounts and $4-$30 for email passwords, according to the Symantec Global Internet Security Threat Report.

## 2.4 Session Guidelines

Here are some general guidelines on sessions.

- *Do not store large objects in a session*. Instead you should store them in the database and save their id in the session. This will eliminate synchronization headaches and it won't fill up your session storage space (depending on what session storage you chose, see below). This will also be a good idea, if you modify the structure of an object and old versions of it are still in some user's cookies. With server-side session storages you can clear out the sessions, but with client-side storages, this is hard to mitigate.
- *Critical data should not be stored in session*. If the user clears their cookies or closes the browser, they will be lost. And with a client-side session storage, the user can read the data.

## 2.5 Session Storage

*Rails provides several storage mechanisms for the session hashes. The most important is `ActionDispatch::Session::CookieStore`.*

Rails 2 introduced a new default session storage, CookieStore. CookieStore saves the session hash directly in a cookie on the client-side. The server retrieves the session hash from the cookie and eliminates the need for a session id. That will greatly increase the speed of the application, but it is a controversial storage option and you have to think about the security implications of it:

- Cookies imply a strict size limit of 4kB. This is fine as you should not store large amounts of data in a session anyway, as described before. *Storing the current user's database id in a session is usually ok*.
- The client can see everything you store in a session, because it is stored in clear-text (actually Base64-encoded, so not encrypted). So, of course, *you don't want to store any secrets here*. To prevent session hash tampering, a digest is calculated from the session with a server-side secret and inserted into the end of the cookie.

That means the security of this storage depends on this secret (and on the digest algorithm, which defaults to SHA1, for compatibility). So *don't use a trivial secret, i.e. a word from a dictionary, or one which is shorter than 30 characters*.

`secrets.secret_key_base` is used for specifying a key which allows sessions for the application to be verified against a known secure key to prevent tampering. Applications get `secrets.secret_key_base` initialized to a random key present in `config/secrets.yml`, e.g.:

```
development:
  secret_key_base: a75d...

test:
  secret_key_base: 492f...

production:
  secret_key_base: <%= ENV["SECRET_KEY_BASE"] %>
```

Older versions of Rails use CookieStore, which uses `secret_token` instead of `secret_key_base`that is used by EncryptedCookieStore. Read the upgrade documentation for more information.